

Software Quality Assurance (SQA)

Chapter 1: Introduction and Software Quality Concepts

Instructor: Prof. Nien-Lin Hsueh (with Gemini AI)

Chapter Highlights

- **1.1 Software Crisis:**
 - Major historical cases (Patriot, NASA, CAL, Disney, Household Registration System) and reflections.
- **1.2 Software Quality and Definitions:**
 - IEEE four software components, Garvin's five quality views, and three levels of quality definitions.
- **1.3 Quality Models:**
 - ISO 9126 six quality characteristics and modern ISO 25010 practical testing tech map.
- **1.4 Quality Control & Quality Assurance (QC vs. QA & CoQ):**
 - QC product inspection vs. QA process prevention, and Cost of Quality (CoQ) model.

1.1 Software Crisis

"We all know the law of conservation of matter; we are even more familiar with the law of conservation of Bugs."

1.1 What is the Software Crisis?

- **Background & Origin:**

- In the late 1960s, hardware computing power grew exponentially, but software development techniques failed to keep pace.
- Software scale and complexity grew exponentially, leading to projects frequently being **delayed, over budget, low quality, or completely failing**.

- **Core Challenges:**

- Software is "invisible and intangible" (intangibility), making progress difficult to measure precisely.
- System complexity exceeds what a single human brain can fully manage.
- Lack of standardized engineering methods, process guidelines, and quality control systems.

Case 1: Patriot Missile Incident (1991)

- **Incident Background:**

- During the 1991 Gulf War, an Iraqi Scud missile hit the US Army barracks in Dhahran, Saudi Arabia, killing **28 soldiers and injuring over 100**.

- **Fatal Defect:**

- The Patriot system's internal clock used a **24-bit floating-point register**, which introduced a tiny truncation/rounding error for every hour of operation.
- After running continuously for over **100 hours** without rebooting, the accumulated time error reached **0.33 seconds**.

- **Disastrous Consequence:**

- A Scud missile travels at Mach 4.2 (1.5 km/s). A delay of 0.33 seconds equates to **about 600 meters of distance offset**. The radar search window failed to lock onto the target, and no interceptor missile was fired.

- **SQA Lesson:** Precision and cumulative error issues; need for long-term continuous running reliability and stress testing (Long-term Reliability Testing).

Case 2: NASA Mars Climate Orbiter (1998)

- **Incident Background:**

- In 1998, NASA launched the "Mars Climate Orbiter" (costing nearly \$200 million), which lost contact and burned up upon arrival in the Martian atmosphere.

- **Fatal Defect:**

- Two collaborating engineering teams used **different measurement units**:
 - Lockheed Martin (contractor): **Imperial units** (pound-force seconds, lbf · s)
 - NASA Jet Propulsion Laboratory (JPL): **Metric units** (Newton seconds, N · s)
- The propulsion control software directly used the imperial values in metric calculations without unit conversion.

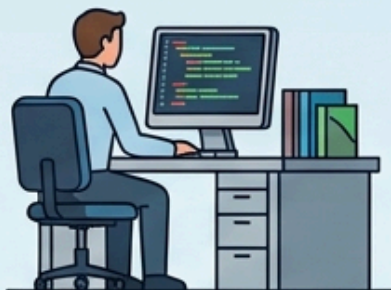
- **Disastrous Consequence:**

- The planned orbital altitude was 140–150 km, but the spacecraft descended to **57 km**, burning up in the thin atmosphere due to friction.

- **SQA Lesson:** Cross-team interface contracts (Interface Contract), unit compatibility validation, and specification reviews.

1998 THE NASA MARS CLIMATE ORBITER SOFTWARE/ SOFTWARE FAILURE DUE TO UNIT MISMATCH

CONTRACTOR (LOCKHEED MARTIN) - SOFTWARE OUTPUT



GROUND SOFTWARE

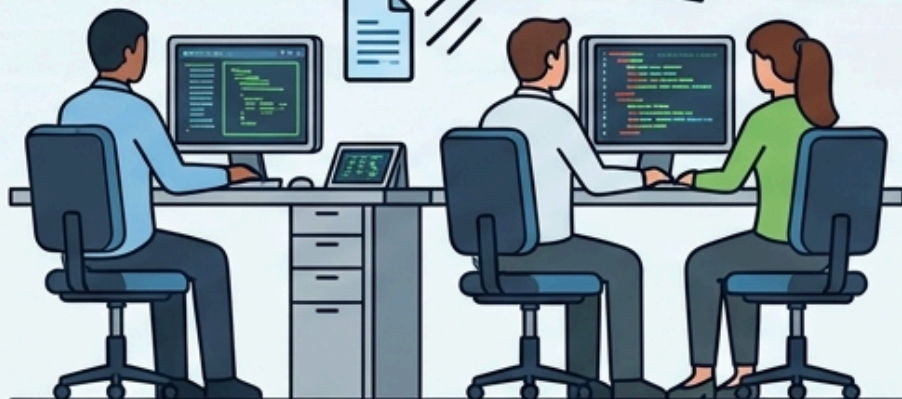
THRUSTER IMPULSE DATA

OUTPUT:
IMPERIAL UNITS:

lbf-s

(Pound-force seconds)

10.00 s
10.00 s



INTERFACE CONTRACT BREAKDOWN



**NO UNIT
CONVERSION!**



**NO UNIT
CONVERSION!**

NASA JPL - SPACECRAFT RECEPTION & OPERATION



NAVIGATION SOFTWARE

EXPECTING
METRIC UNITS:

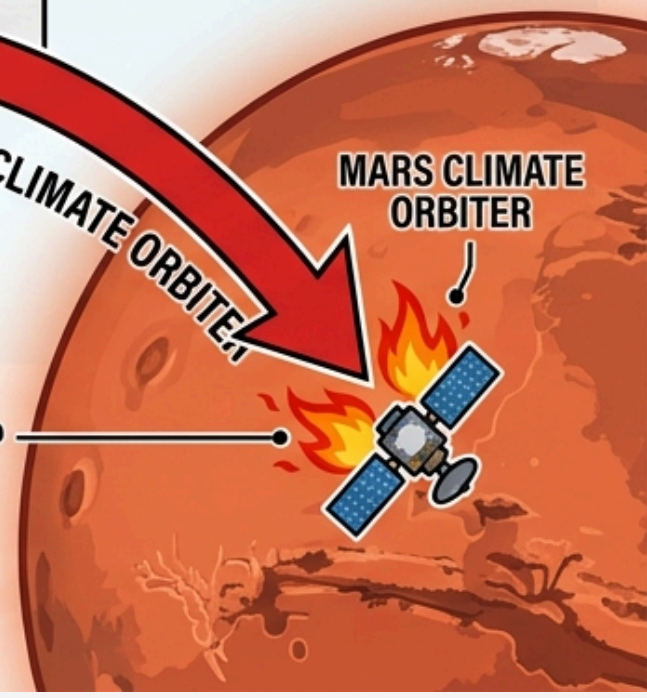
N-s

(Newton-seconds)

MARS CLIMATE ORBITER

TRAJECTORY DROP

CRITICAL ALTITUDE: ~57 km
SPACECRAFT BURNS UP IN ATMOSPHERE



Case 3: China Airlines Nagoya Crash (1994)

- **Incident Background:**

- In 1994, China Airlines flight CI140 (Airbus A300-622R) crashed during landing at Nagoya Airport, killing **264 people**.

- **Fatal Defect:**

- The co-pilot accidentally triggered the "**Go-Around**" mode during approach.
- The pilot tried to manually push the nose down to land; however, the flight control computer, locked in Go-Around mode, aggressively trimmed the tail horizontal stabilizer up to raise the nose.

- **Disastrous Consequence:**

- **Man and machine fought for control**, causing the aircraft to pitch up too steeply, stall, and crash. Airbus subsequently modified the A300 flight control logic globally.

- **SQA Lesson:** Human-Machine Interface (HMI/UX) status transparency, feedback on abnormal operations, and priority/override arbitration between manual control and automation.

1994 CHINA AIRLINES NAGOYA CRASH: FLIGHT CONTROL SOFTWARE CONFLICT (HMI MODE CONFUSION)

Case Study: Human-Machine Interface Design & System Safety

PILOT ACTIONS: MANUAL DESCENT



CONFLICT: HMI MODE CONFUSION & CONTROL FIGHT

Human tries to push plane nose-down while Autopilot forces it nose-up.

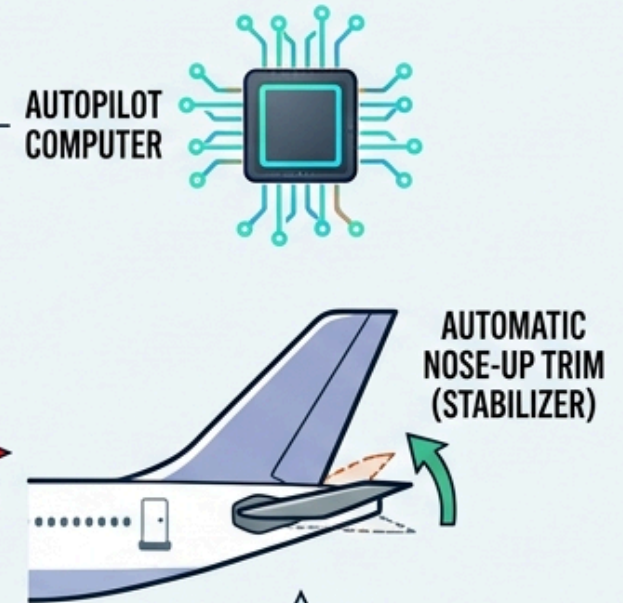
MODE CONFUSION:
Human unaware of conflicting TOGA mode command.



EXTREME PITCH-UP & AERODYNAMIC STALL

LOSS OF LIFT → → LOSS OF CONTROL

AUTOPILOT ACTIONS: AUTOMATIC CLIMB



AUTOPILOT
COMPUTER



AUTOMATIC
NOSE-UP TRIM
(STABILIZER)

AUTOPILOT IN GO-AROUND (TOGA)
MODE: COMMANDING NOSE-UP.



ATTITUDE INDICATOR

ALTOMETER

Case 4: Disney's "The Lion King" Compatibility Crisis (1994)

- **Incident Background:**

- On Christmas 1994, Disney released "The Lion King" PC game, heavily promoted alongside Compaq PCs. Thousands of families eagerly anticipated playing it.

- **Fatal Defect:**

- The game used Microsoft's newly released WinG graphics library but was only tested on a very narrow set of hardware. **Broad compatibility testing was not performed.**
- On many home PCs, launching the game led to blue screens or immediate crashes.

- **Disastrous Consequence:**

- Customer service lines were overwhelmed by angry parents on Christmas Day, dealing a major blow to the brand.

- **SQA Lesson:** Diverse **compatibility testing** across hardware/OS environments, which eventually prompted Microsoft to develop the standardized DirectX API.

Case 5: Taiwan Public System Incidents

- **2014 Household Registration System Failure:**

- On day one, the system suffered severe lag island-wide, preventing citizens from getting transcripts or IDs.
- Core Issues: Hardware-software architecture compatibility, insufficient load testing, and unoptimized database queries.

- **2021 High School Learning Portfolio Loss:**

- An outsourced engineer made an operational error during VM migration and reboot, deleting 25,000 students' data.
- Core Issues: Configuration Management failures, lack of verified backup and disaster recovery processes.

- **2014 Freeway Electronic Toll Collection (ETC):**

- Frequent double-billing and phantom charges during early rollout highlighted extreme boundary and accuracy testing requirements in high-volume, real-time transaction systems.

Reflections on the Software Crisis

- **"Software and cathedrals are much the same — first we build them, then we pray."** — — *Sam Redwine*
- Software quality is not just about "finding bugs after writing code"; it involves:
 - **Clarity and correctness of requirements and specifications**
 - **Robustness and health of architectural design**
 - **Cross-team communication, standards, and contract compliance**
 - **A comprehensive lifecycle testing and quality assurance framework**
- SQA aims to use engineering methods to transform crises into controlled quality.

Concept Check Question (CCQ 1)

What was the fundamental software cause of the Patriot Missile system (1991) intercept failure at Dhahran?

- **A.** A communication network outage prevented the radar from sending commands to the launcher.
- **B.** A floating-point rounding error in the 24-bit clock register accumulated to 0.33 seconds after 100 hours of continuous operation.
- **C.** A memory leak in the code caused the operating system to crash.
- **D.** The radar algorithm misidentified a US fighter jet as an enemy Scud missile.



Concept Check Question (CCQ 2)

What is the most important lesson for software engineers from the NASA Mars Climate Orbiter (1998) crash?

- **A.** Multi-threading must be used to prevent computational blocking.
- **B.** Interface specifications (such as measurement units) between modules across systems or teams must be strictly defined and validated.
- **C.** Space mission software must not use any third-party libraries.
- **D.** As long as engine thrust is sufficient, small software calculation errors will not affect orbit.



Concept Check Question (CCQ 3)

What was the critical design flaw in the flight control software and human-machine interaction in the 1994 China Airlines Nagoya crash?

- **A.** A computer virus on board caused the control panels to go black.
- **B.** A lack of control override arbitration and clear status indications when manual control conflicted with the computer's Go-Around mode.
- **C.** A divide-by-zero exception occurred in the engine fuel calculation formula.
- **D.** The autopilot system failed to implement altitude sensing.



1.2 Software Quality and Definitions

"People forget how fast you did a job, but they remember how well it was done." — — Howard Newton

1.2.1 What is Software?

- The broad IEEE definition of **Software**:

*Computer **programs**, **procedures**, and possibly associated **documentation** and **data** pertaining to the operation of a computer system.*

- Software is not just executable code; it comprises:
 - **Programs**: Instructions and algorithms.
 - **Procedures**: Operational manuals, deployment, and maintenance rules.
 - **Documentation**: Requirements specs, architecture design, test cases.
 - **Data**: Initialization parameters, configuration files, test datasets.

EDUCATIONAL SOFTWARE ENGINEERING INFOGRAPHIC: 4 CORE ELEMENTS OF SOFTWARE (IEEE STANDARD)



PROGRAMS

Source code and compiled binaries

- SOURCE CODE
- COMPILED BINARIES
- EXECUTABLES
- SCRIPTS



PROCEDURES

CI/CD automated pipelines, deployment workflows

- CI/CD AUTOMATED PIPELINES
- DEPLOYMENT WORKFLOWS
- BUILD SCRIPTS
- OPERATIONAL GUIDELINES



DOCUMENTATION

SRS specifications, OpenAPI contracts, test plans

- SRS SPECIFICATIONS
- OPENAPI CONTRACTS
- TEST PLANS & REPORTS
- USER MANUALS



DATA

Configuration files, migration scripts, database fixtures

- CONFIGURATION FILES
- MIGRATION SCRIPTS
- DATABASE FIXTURES
- INITIALIZATION DATA

Concept Check Question (CCQ 4)

According to the IEEE definition and course material, which of the following does NOT fall under "software"?

- **A.** System design documents and test cases created during development.
- **B.** System initialization data required to execute programs.
- **C.** Operating procedures and workflows followed when installing and running the system.
- **D.** Only the compiled binary machine code executing on the server, excluding other items.



1.2.2 David Garvin's Five Quality Views

Quality View	Core Meaning	Software Example
Transcendental	Cannot be measured directly, but exquisite quality is felt upon experience	Smooth, elegant UI/UX and attention to details
User view	Fitness for use; meeting user needs and expectations	Usability, feature solving pain points
Manufacturing	Conformance to engineering specs and standard processes	Adhering to standards, Zero Bug target
Product view	Inherent technical characteristics and internal design quality	High cohesion, low coupling, clean code
Value-based	Affordability and price-performance ratio (ROI)	Business value brought, willingness to subscribe

DAVID GARVIN'S 5 VIEWS OF PRODUCT QUALITY (SOFTWARE ENGINEERING)

TRANSCENDENTAL VIEW

1. Transcendental View



Aesthetic elegance and UX delight

- Intangible excellence
- Intuitive & beautiful design
- Delighting the user experience
- Emotional appeal & perception

USER VIEW

2. User View



Fitness for use and solving pain points

- Meets user needs & expectations
- Solves key problems effectively
- Ease of learning & usability
- Fits into the user's workflow

MANUFACTURING VIEW

3. Manufacturing View



Conformance to specifications and standards

- Defect-free production
- Adherence to technical specifications
- Consistency and repeatability
- Reliability and process standards

PRODUCT VIEW

4. Product View



Internal code structural attributes and modularity

- Code quality & maintainability
- Modularity and scalability
- High cohesion, low coupling
- Architectural soundness

VALUE-BASED VIEW

5. Value-based View



ROI and business cost-effectiveness

- Optimal value for the price
- Maximizing business benefits (ROI)
- Balanced cost & performance
- Strategic market differentiation

Concept Check Question (CCQ 5)

A system perfectly meets every functional requirement in the contract, but its architecture is messy and extremely difficult to maintain or extend. Under Garvin's views, which view would deem this software of poor quality?

- **A.** Manufacturing View
- **B.** Product View and professional implicit quality
- **C.** Legal Contract View
- **D.** Outsourcing Billing View



1.2.3 Three Levels of Software Quality Definition

1. **Conformance to Specifications** (*Crosby, 1979*):

- Meeting the written requirements specification. (*Flaw: Specs are often incomplete*)

2. **Fitness for Use** (*Juran, 1998*):

- Satisfying users' and stakeholders' expectations and actual needs.

3. **Conformance to Professional Standards & Implicit Characteristics** (*Pressman*):

- Beyond explicit features, this includes implicit professional attributes like **maintainability, security, and robustness**.

Concept Check Question (CCQ 6)

Which statement regarding the definition of software quality best aligns with Pressman's view on professionally developed software?

- **A.** As long as the program runs without bugs, it is high-quality software.
- **B.** Software quality depends solely on whether it fully satisfies the functional requirements defined in the spec.
- **C.** Software quality includes not only explicit functional and performance requirements and explicit development standards, but also implicit characteristics expected of professional software (such as maintainability, readability).
- **D.** Software quality depends entirely on subjective user satisfaction, regardless of development processes or documentation.



1.3 Quality Models (ISO 9126 / ISO 25010)

"Quality models serve as the compass of software engineering, defining what good software is."

DIFFERENT INDUSTRIES & PRODUCTS HAVE UNIQUE QUALITY MODELS

1

AUTOMOBILE



Prioritizing

- SAFETY
- MAINTAINABILITY
- REPAIRABILITY

2

LUXURY MECHANICAL WATCH



Prioritizing

- PRECISION
- CRAFTSMANSHIP
- TRANSCENDENTAL ELEGANCE

3

FAST FOOD RESTAURANT



Prioritizing

- SPEED
- CONSISTENCY
- VALUE

4

SOFTWARE SYSTEMS



Prioritizing

- SCALABILITY
- SECURITY
- PORTABILITY
- FAULT TOLERANCE

1.3.1 ISO 9126 Six Quality Characteristics

- **Functionality:** Suitability, accuracy, interoperability, compliance, security.
- **Reliability:** Maturity (MTBF/MTTF), fault tolerance, recoverability.
- **Usability:** Understandability, learnability, operability, attractiveness.
- **Efficiency:** Time behavior (response time), resource utilization (CPU/RAM/I-O).
- **Maintainability:** Analyzability, changeability, stability, testability.
- **Portability:** Adaptability, installability, co-existence, replaceability.

1.3.2 Modern ISO 25010 and Practical Testing Tech Map

ISO 25010 Quality Characteristic	Core Guardian Technology (Course Focus)
Functional Suitability	Equivalence Partitioning (EP), Boundary Value Analysis (BVA), JUnit 5, BDD (Cucumber)
Reliability	Assertions, Property-Based Testing (jqwik) , Chaos Engineering
Maintainability	Static Analysis (SonarQube/SpotBugs), Mutation Testing (PITest)
Security	Static Application Security Testing (SAST), Fuzz Testing (Fuzzing)
Performance Efficiency	High-Concurrency Load Testing with k6 / Apache JMeter , Memory Profiling
Compatibility	Microservice Contract Testing (Pact) , Cross-version Compatibility Testing
Portability	Containerized Testing with Testcontainers , Docker Environment Validation
Usability	Playwright E2E Acceptance Testing , UI Automation Workflows

ISO 25010 PRODUCT QUALITY CHARACTERISTICS

(Focusing only the first level)



1. Functional Suitability

Meets stated and implied needs



2. Reliability

Maintains performance over time



3. Performance Efficiency

Optimizes response times and resources



4. Usability

Easy to learn, understand, and operate



5. Security

Protects information and assets



6. Maintainability

Can be modified, tested, or repaired efficiently



7. Portability

Adaptable to different environments



8. Compatibility

Interacts seamlessly with other systems

Concept Check Question (CCQ 7)

A server automatically reconnects after a network disconnect without losing in-progress transaction data. Under ISO 9126, which quality characteristic does this represent?

- **A.** Fault tolerance and recoverability under Reliability.
- **B.** Installability under Portability.
- **C.** Attractiveness under Usability.
- **D.** Compliance under Functionality.



Concept Check Question (CCQ 8)

Modifying the field length in a personnel module of a software system accidentally causes a completely unrelated financial settlement module to fail. Which characteristic in the ISO 9126 model does this indicate the system is performing poorly in?

- **A.** Analyzability
- **B.** Stability
- **C.** Fault tolerance
- **D.** Interoperability



Concept Check Question (CCQ 9)

In the ISO 9126 quality model, when we evaluate "the ease of transferring a software system from one hardware, software, or execution environment to another", which quality characteristic are we evaluating?

- A. Functionality
- B. Maintainability
- C. Portability
- D. Efficiency



1.4 Quality Control (QC) and Quality Assurance (QA)

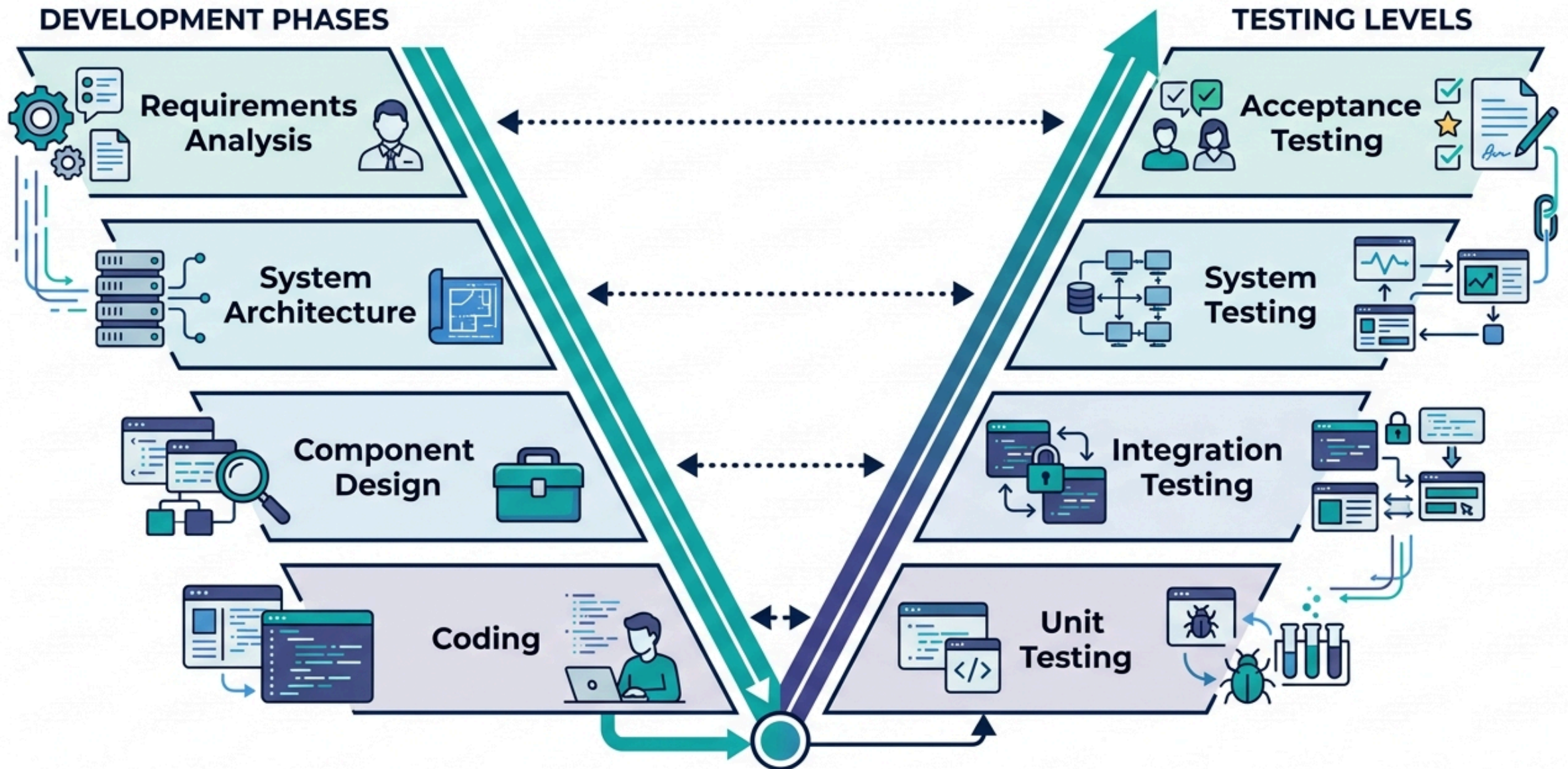
1.4.1 Essential Differences: QC vs. QA

- **Quality Control (QC):**
 - **Product-oriented**
 - Post-inspection, finding bugs (Defect Detection).
 - Major activities: Unit testing, system testing, product output reviews.
- **Quality Assurance (QA):**
 - **Process-oriented**
 - Prevention, process improvement (Defect Prevention).
 - Major activities: Defining development standards, code review guidelines, CI/CD pipelines, quality audits.

1.4.2 The V-Model: Development & Testing Symmetry

- **Verification (Left side) vs. Validation (Right side).**
- Establishes a parallel mapping between early development phases and corresponding testing levels:
 - **Requirements Analysis → Acceptance Testing**
 - **System Architecture → System Testing**
 - **Component Design → Integration Testing**
 - **Coding → Unit Testing**
- **Core Value:** Design test cases early before writing the actual code to prevent defects from leaking.

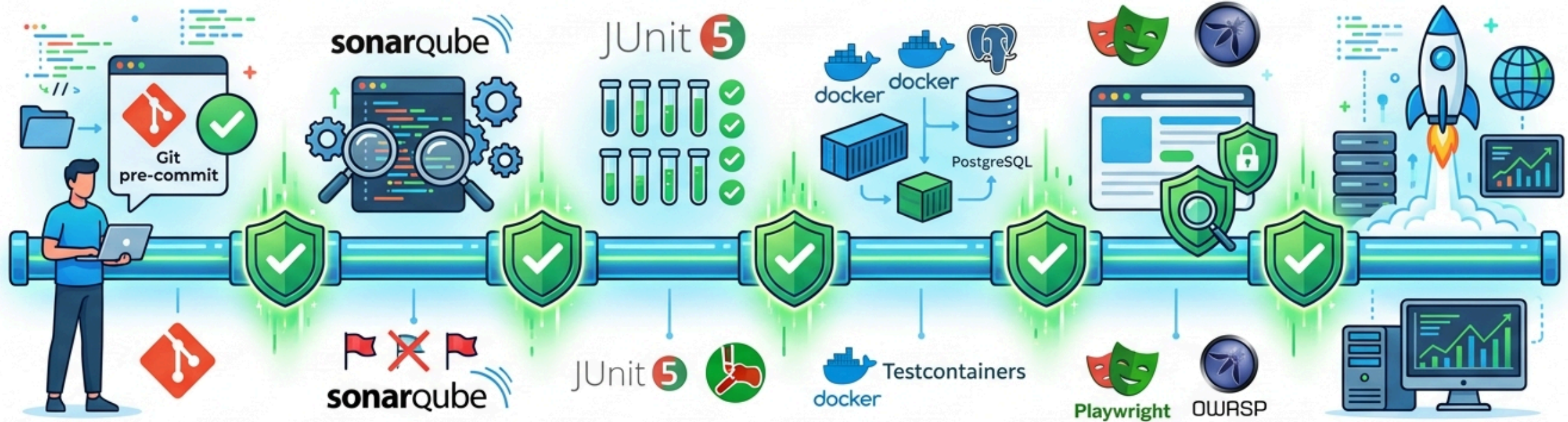
THE V-MODEL: INTEGRATING DEVELOPMENT AND TESTING



1.4.3 Modern Agile & DevOps Continuous Quality Gates

- **Continuous Quality Gates:** Automate quality checks at each step in the CI/CD pipeline.
- **The Six Key Quality Gates:**
 1. **Code Commit Gate:** Pre-commit hooks check formatting and syntax.
 2. **SAST Code Quality Gate:** SonarQube scans for code smells and OWASP bugs.
 3. **Unit Tests & Coverage Gate:** JUnit 5 tests and JaCoCo coverage (e.g. >80%).
 4. **Integration Tests Gate:** Testcontainers spin up real databases/caches.
 5. **E2E & Security Gate:** Playwright UI workflows and dynamic security scanning.
 6. **Production Gate:** Canary deployments and observability monitoring.

CONTINUOUS QUALITY GATES IN MODERN DEVOPS CI/CD PIPELINE



1. CODE COMMIT

Local validations & Pre-commit Hooks (e.g., husky)

2. SAST & CODE QUALITY

Analyze for vulnerabilities, code code smells, technical debt. SonarQube Gate

3. UNIT TESTS & COVERAGE

Validate individual components. Meet required JaCoCo Code Coverage (e.g., >80%)

4. INTEGRATION TESTS

Verify integration points in isolated containers (Testcontainers)

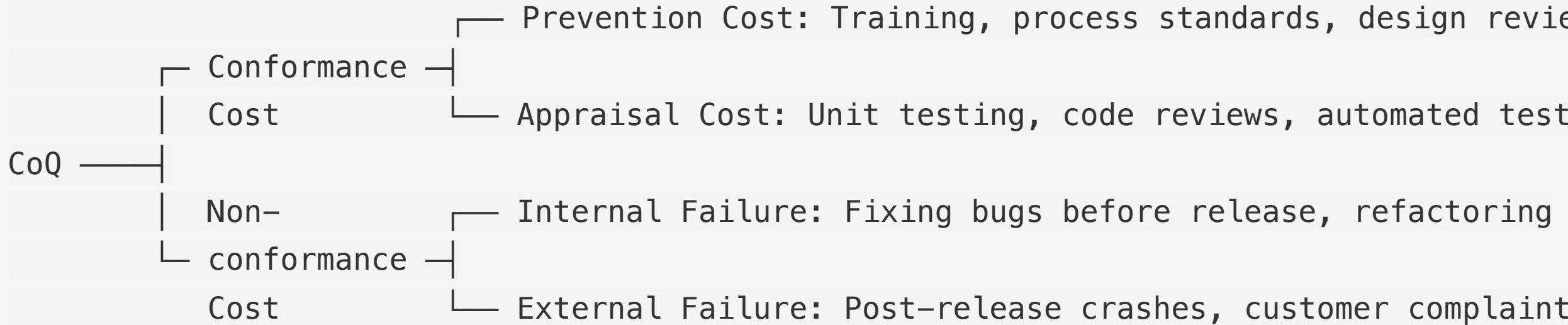
5. E2E & SECURITY SCAN

End-to-End browser tests (Playwright). Staging environment checks & Security Scanning

6. PRODUCTION & OBSERVABILITY

Canary Deploy, blue-green release. Real-time Monitoring & Feedback loop

1.4.4 Cost of Quality (CoQ)



- **1:10:100 Rule:**

- Fixing a bug during requirements phase costs **\$1**.
- Fixing it during development/testing costs **\$10**.
- Fixing it after product release costs **\$100+** and damages brand reputation!

UNDERSTANDING COST OF QUALITY (CoQ)

CONFORMANCE COSTS (Investing in Quality)

PREVENTION COSTS (Build it Right)

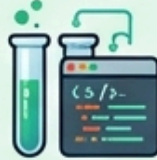


Architecture &
Design Review



Contract
Design

APPRAISAL COSTS (Checking Quality)



Unit
Testing



Static
Analysis

NON-CONFORMANCE COSTS (Failure to build quality)

INTERNAL FAILURE COSTS (Caught Before Release)

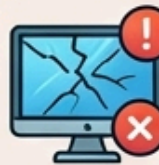


Bug
Tracking



Debugging
& Rework

EXTERNAL FAILURE COSTS (Found After Release)



Production
Crashes

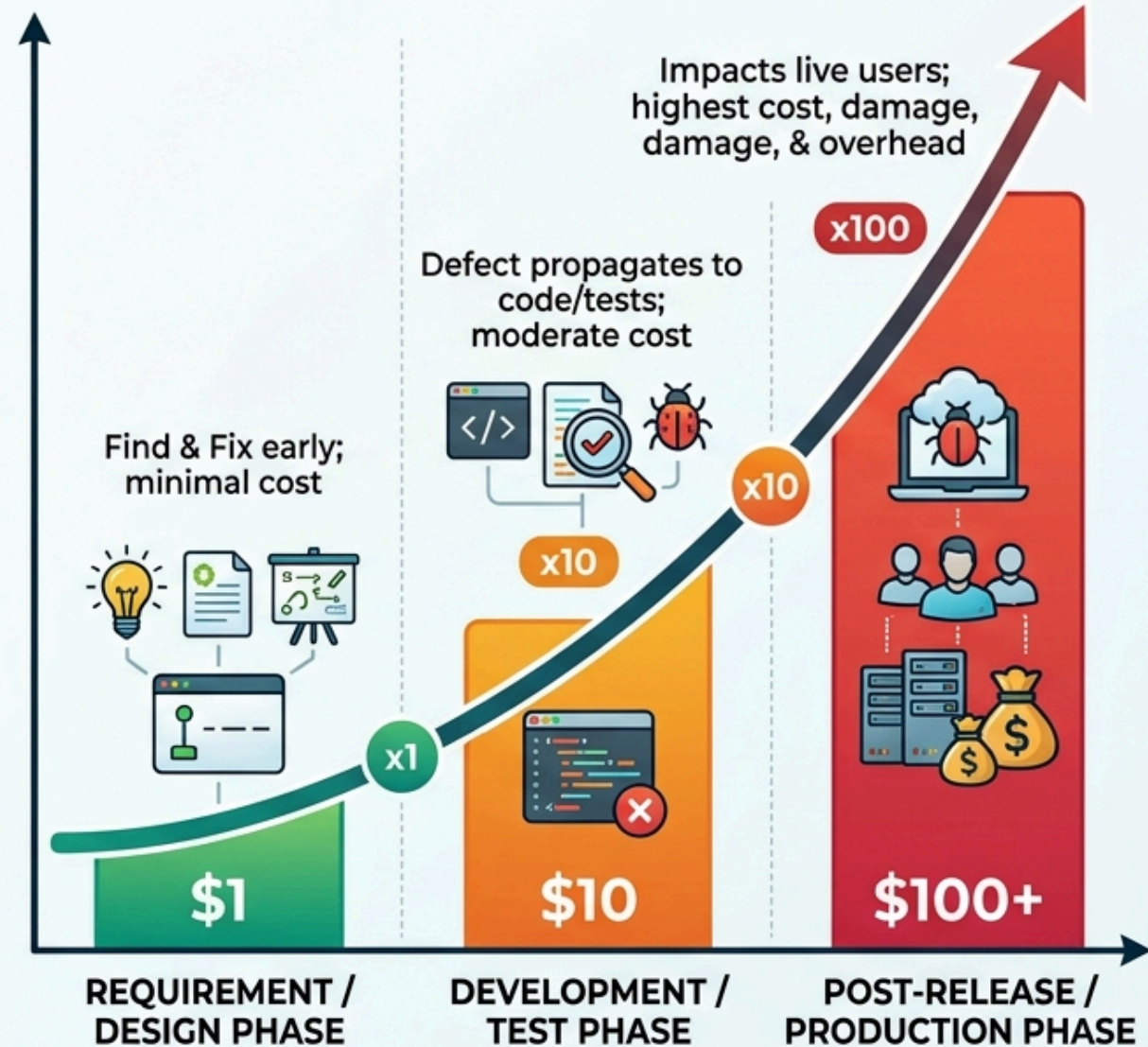


Customer
Support



Lost
Business

THE 1:10:100 RULE OF TENS (Cost to Fix a Defect)



EARLY PREVENTION IS KEY: Prioritize Conformance to Avoid Escalating Non-Conformance Costs.

Concept Check Question (CCQ 10)

Introducing automated static code analysis (e.g. SonarQube) and engineer quality training courses fall under which categories in the Cost of Quality (CoQ) model respectively?

- **A.** Internal Failure Cost, External Failure Cost
- **B.** Appraisal Cost, Prevention Cost
- **C.** External Failure Cost, Appraisal Cost
- **D.** Liability Cost, Maintenance Cost



Chapter Highlights Review

Summary and Highlights

- **Software Crisis:** Historical cases (Patriot, NASA, CAL, Disney) show that software defects can be fatal and extremely costly.
- **Software Quality:** Software comprises programs, procedures, documentation, and data; Garvin's five quality views offer diverse perspectives.
- **ISO 9126:** Six core quality characteristics (Functionality, Reliability, Usability, Efficiency, Maintainability, Portability).
- **QA vs. QC:** QA focuses on process and prevention, QC focuses on product and inspection; earlier prevention means lower costs (1:10:100 rule).

Q & A

Thank you!